# CodeMentor AI

**Un motor híbrido de mentoría de código que audita y formula preguntas socráticas sin corregir el código.**

PYTHON 3.12 | 3.700+ PRUEBAS | MYPY --STRICT | SIN DOCKER | LICENCIA MIT

## De un Vistazo

| Característica | Descripción |
|---|---|
| **Lenguajes y Entornos** | C, C++, C#, Python, HTML, CSS + contextos de Unity y Unreal Engine |
| **Reglas Estáticas** | 41 reglas deterministas (sintaxis, fugas de memoria, bucles Big-O, seguridad) |
| **Prevalidación** | tree-sitter (Tier 0) + gcc / clang / Roslyn (Tier 1 validación sintáctica) |
| **Modelos LLM** | Claude Code CLI, Codex, Anthropic API, Gemini, Ollama (local) |
| **Interfaces** | Escritorio Windows (.exe), Web React + Vite, CLI, API REST + SSE |
| **Evaluación y Métricas** | 166 casos etiquetados (dev 96 / test 70), 0% fuga de código, 100% anti-inyección |

## ¿Por qué CodeMentor AI?

Cuando un estudiante pregunta a un LLM "¿qué está mal en mi código?", el modelo suele proporcionar la solución lista para copiar. El estudiante entrega la tarea sin comprender el error; el aprendizaje se anula. CodeMentor AI revierte este flujo:

- **Preguntas en lugar de soluciones:** Las respuestas de LLM nunca van directas al usuario; un filtro contra fugas (Leakage Guard) bloquea soluciones completas (0% fuga).
- **Base determinista:** En errores sintácticos el sistema sale de inmediato (~1 ms) sin consumir tokens LLM. Ante caídas de red, degrada elegantemente a análisis estático.
- **Pistas progresivas (1 → 3):** Nivel 1 (Pista conceptual) → Nivel 2 (Línea y contexto) → Nivel 3 (Orientación). Jamás se entrega la respuesta final.
- **Seguridad y Aislamiento:** El código del usuario se procesa como dato no confiable mediante stdin en subprocesos aislados con límites estrictos.

## Qué Experimenta el Estudiante

**1. Error de Sintaxis (Salida Temprana, 0 Tokens LLM, ~1 ms)**

```
int total = 0 // falta punto y coma
```

**Pregunta del Mentor:** "Una instrucción en la línea 4 parece no finalizar con ';'. ¿Qué espera encontrar el compilador tras esta declaración?"

**2. Fuga de Memoria y Regla Estática (C-MEM-001)**

```
int *tmp = malloc(n * sizeof(int)); // nunca se libera
```

**Pregunta Socrática (Nivel 1):** "¿Qué ocurre con la memoria dinámica solicitada una vez que la función retorna, y quién tiene la responsabilidad de liberarla?"

## Métricas de Ingeniería y Evaluación

| Métrica | Claude Opus (two_pass) | Solo Estático (Sin LLM) |
|---|---|---|
| Auditoría Precisión / Recall / F1 (dev 96) | **79 / 97 / 87** | 100 / 57 / 73 |
| Precisión / Recall en Salida Temprana | 100% / 100% | 100% / 100% |

| | | |
|---|---|---|
| Exactitud en Complejidad Big-O | 97% / 97% | 97% / 97% |
| Fuga de Solución (Leakage) | **0%** | — |
| Resistencia a Inyección de Prompts | **100%** | — |